MCP for an FDE (Forward Deployed Engineer)

Complete Beginner-Friendly Guide

For an FDE, understanding MCP is important because you build AI applications that connect LLMs to real business systems, databases, internal tools, and APIs.

Think of a company building an AI assistant for its employees. The assistant should be able to check customer information, create tasks, retrieve documents, and query databases. MCP provides a standard way for AI applications to discover and use those capabilities.

Let's understand every topic with simple Python examples and real-world FDE use cases.

1. Why MCP was needed

MCP stands for **Model Context Protocol**. It is an open protocol that standardizes how AI applications connect to external tools and data sources.

The problem before MCP

Suppose you build an AI assistant that connects to three systems:

- A CRM API to retrieve customer details.
- A database to check orders.
- A task management application to create tasks.

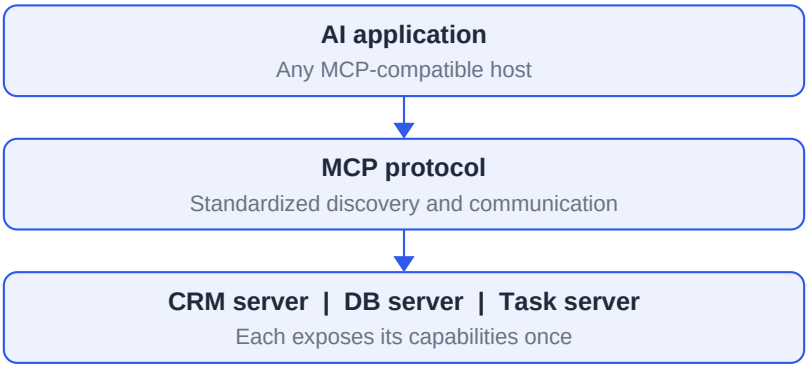
Without a shared protocol, developers may need to implement different integration logic for each AI application and each system.

Before MCP: custom integrations

AI applications	Integrations you must build
AI App A, AI App B	Custom CRM code, custom database code (each app wires up each system separately)

Each integration may require separate schemas, authentication handling, and error handling.

After MCP



Why it matters for an FDE:

- Reuse integrations across compatible AI applications.
- Discover available tools dynamically.

- Standardize how tools receive inputs and return results.
- Connect LLMs to existing enterprise systems.
- Reduce repeated integration code.

> MCP does not eliminate all custom integration work. You still need to build or configure the server, handle authentication, and enforce permissions.

2. MCP vs Tool Calling

This is one of the most important interview questions.

Tool calling is a capability of an LLM: the model can request that an application execute a function.

MCP is a protocol that helps an application discover and communicate with external capabilities.

Feature	Tool calling	MCP
Main purpose	Let the model request function execution	Standardize access to tools and context
Tool definitions	Provided to the model by the application	Can be discovered from an MCP server
Execution	Usually handled by application code	Handled by the client/server integration
Standard protocol	Depends on the LLM/API implementation	Defines a common protocol
Can work without the other?	Yes	Yes

Example

A user asks: "Create a task to call the customer tomorrow." The LLM generates a tool call:

```
{
  "name": "create_task",
  "arguments": {
    "title": "Call the customer",
    "due_date": "2026-10-11"
  }
}
```

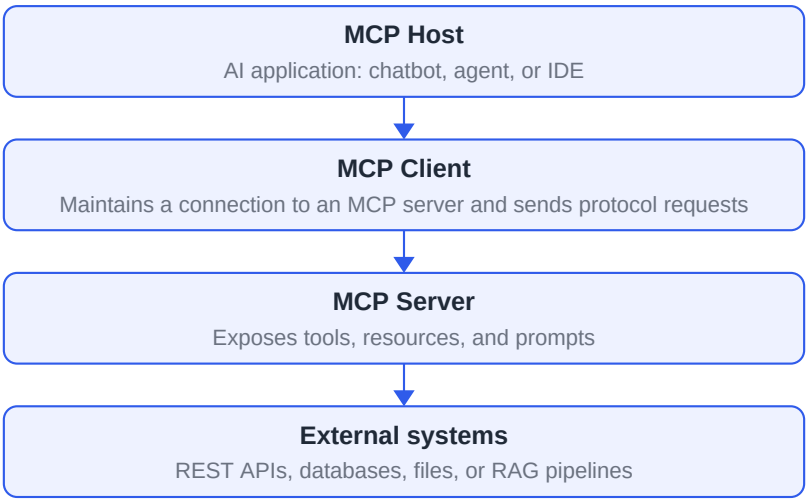
This is a tool-call request, not the task being created yet.

Your application must execute the function. With MCP, the application can discover the server's `create_task` tool and invoke it using the MCP protocol.

> **Remember:** MCP is not an LLM, and tool calling is not a protocol that replaces MCP. They work together.

3. MCP Host, Client, and Server architecture

MCP has three main components: Host, Client, and Server.



What does each component do?

- **Host:** The application that manages the LLM, conversation, user interface, and MCP connections.
- **Client:** The component inside the host that communicates with an individual MCP server.
- **Server:** The component that exposes capabilities and executes requests against the underlying system.

For example, in a customer-support AI application:

- Host = the customer-support chatbot.
- Client = the MCP connection manager inside the chatbot.
- Server = the customer-data MCP server.
- External system = the company's CRM database.

One host can connect to multiple MCP servers, typically using a separate client connection for each server.

4. What an MCP Server actually does

An MCP server is an adapter between an AI application and an underlying system. It can:

1. Advertise the tools it supports.
2. Describe tool inputs using schemas.
3. Execute a requested tool.
4. Return results in a standardized format.
5. Expose resources and reusable prompts.
6. Apply application-level validation, authorization, and error handling.

Imagine a task management system with a database table called `tasks`. The server may expose these tools:

Tool	Purpose
`create_task`	Create a task
`get_task`	Retrieve one task
`list_tasks`	List tasks
`complete_task`	Mark a task complete

The server translates a request like `create_task(title="Prepare report")` into the appropriate database operation or API call.

> **Important:** MCP does not automatically connect to every database or API. You write or configure the server's integration logic.

5. Tools, Resources, and Prompts

MCP provides three important capability types.

Type	What it is	Examples / typical use
Tools (actions)	Let the AI application request an operation.	`create_task()`, `search_customer()`, `run_sql_query()`. Change data, perform calculations, retrieve results, or call services.
Resources (context or data)	Expose information that the application can read.	`file:///reports/sales.csv`, a project configuration, or a customer record. Provide documents, reference data, and context to the AI.
Prompts (reusable instructions)	Provide predefined templates or workflows.	A customer-support response template or a SQL-analysis prompt. Help users and applications start a consistent workflow.

A simple way to remember:

- **Tool:** Do something.
- **Resource:** Read something.
- **Prompt:** Guide how a task should be approached.

These are conceptual distinctions; the exact way the host presents or uses each capability depends on the application.

6. How Tool Discovery works

Tool discovery means the AI application learns which tools a server offers, rather than requiring every tool to be hardcoded into the client. Suppose a task server supports:

```
{
  "name": "create_task",
  "description": "Create a new task",
  "inputSchema": {
    "type": "object",
    "properties": {
      "title": { "type": "string" },
      "priority": {
        "type": "string",
        "enum": ["low", "medium", "high"]
      }
    },
    "required": ["title"]
  }
}
```

The client learns:

- Tool name: `create_task`
- Purpose: create a task

- Required input: `title`
- Optional input: `priority`
- Allowed priorities: `low`, `medium`, `high`

The application can then make these details available to its LLM. The general process is:

1. The host establishes a connection to the MCP server.
2. The client initializes the MCP session.
3. The client sends a tool-discovery request.
4. The server returns tool names, descriptions, and input schemas.
5. The host maps those definitions into the LLM's supported tool-calling format.
6. The LLM can select a tool when the user's request requires it.

Discovery can be repeated when capabilities change or refreshed according to the host's design.

7. `tools/list` and `tools/call`

These are two core MCP protocol operations.

`tools/list`

Retrieves the tools available from an MCP server. Example request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Example response, simplified:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "list_tasks",
        "description": "List all tasks",
        "inputSchema": {
          "type": "object",
          "properties": {}
        }
      }
    ]
  }
}
```

`tools/call`

Requests execution of a discovered tool. Example request:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "create_task",
    "arguments": {
      "title": "Prepare interview"
    }
  }
}
```

The server validates and executes the operation, then returns a result. The important distinction:

- `tools/list` = discover what can be done.
- `tools/call` = request that something be done.

The JSON-RPC examples above are simplified representations of the MCP protocol messages; SDKs usually construct them for you.

8. How tool schemas connect with Structured Output

This is an important topic for AI engineering interviews.

A **schema** defines the expected structure and types of data. **Structured output** is a way of constraining an LLM's response to a particular format, often JSON.

Consider the user request: "Create a high-priority task to prepare my interview." The tool schema might define:

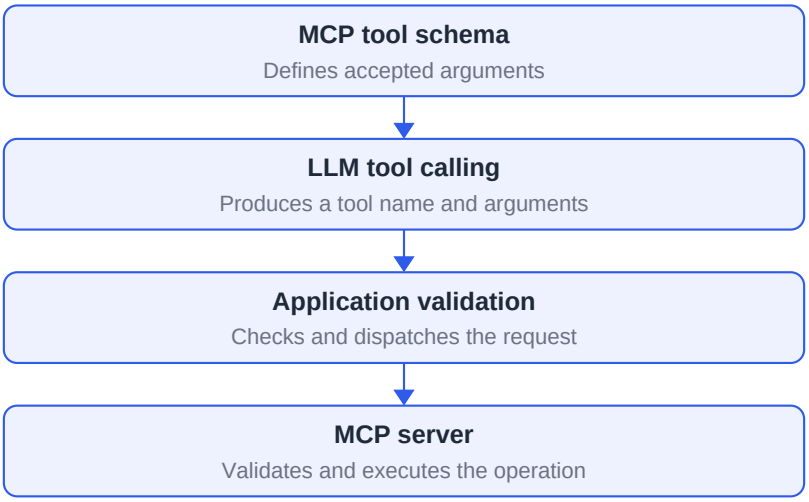
```
{
  "type": "object",
  "properties": {
    "title": {"type": "string"},
    "priority": {
      "type": "string",
      "enum": ["low", "medium", "high"]
    }
  },
  "required": ["title", "priority"]
}
```

The model can generate a structured tool-call request:

```
{
  "title": "Prepare my interview",
  "priority": "high"
}
```

The application then passes these arguments to the MCP server.

How the pieces fit together



There are two related but different schemas:

- **Tool input schema:** Describes arguments accepted by an MCP tool.
- **Structured response schema:** Describes the format required for an LLM's final response or another application's output.

> MCP tool schemas do not automatically guarantee that the model always produces valid arguments, nor do they guarantee that a database operation succeeds. Validate inputs and handle errors on the server.

9. MCP vs REST APIs

REST APIs and MCP solve related but different problems.

Feature	REST API	MCP
Primary purpose	Expose application endpoints	Expose tools, resources, and prompts to AI applications
Discovery	Often documented with OpenAPI or other documentation	Standard tool discovery through `tools/list`
Request style	HTTP methods and URLs	Protocol methods such as `tools/list` and `tools/call`
Data access	Defined by the API	Resources and tools can expose data
AI integration	Application writes API-calling logic	Standardized client/server integration
Can use HTTP?	Yes	Yes, through supported transports such as Streamable HTTP

Example: Weather API

A REST API might provide:

```
GET /weather?city=Bengaluru
```

An MCP server might expose:

```
get_weather(city="Bengaluru")
```

The MCP tool can internally call the existing REST endpoint.

> **Key point:** MCP does not replace REST. MCP can make existing REST services easier for AI applications to discover and use.

10. How MCP works with existing APIs, databases, and RAG systems

This is where MCP becomes especially useful for an FDE.

Source	How MCP fits
A. Existing APIs	An MCP server wraps an API such as a CRM, payment-status service, or ticketing system. The server handles API requests, credentials, and response transformation.
B. Databases	An MCP server can expose safe tools such as `get_customer` or `list_orders`. The underlying implementation runs parameterized SQL or approved queries.
C. RAG systems	An MCP server can expose a `search_knowledge_base` tool that queries a vector database or existing RAG pipeline and returns relevant passages to the AI.

Real-world example: company knowledge assistant

User asks: "Find our leave policy and tell me how many leave days I have remaining." The assistant may need two MCP tools:

1. `search_knowledge_base(query)` retrieves the leave policy through a RAG pipeline.
2. `get_employee_leave_balance(employee_id)` retrieves the user's balance from an HR system.

The LLM combines the retrieved information into a response. A typical architecture:

```
User
  |
AI Application + LLM
  |
MCP Client
  |
  +-- HR MCP Server ----------> HR REST API / Database
  |
  +-- Knowledge MCP Server ---> RAG Pipeline --> Vector DB
  |
Answer with relevant information
```

For production systems, the FDE must also enforce user-specific authorization, data filtering, audit logging, and protection against sensitive information being exposed.

11. Building a simple Task MCP Server in Python

Let's build a beginner-friendly server using the official Python MCP SDK.

Step 1: Install the SDK

```
pip install "mcp[cli]"
```

Step 2: Create task_server.py

This example stores tasks in memory, so it is easy to understand. Tasks disappear when the server restarts.

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Task Server")

tasks = []
next_id = 1


@mcp.tool()
def create_task(title: str, priority: str = "medium") -> dict:
    """Create a new task."""
    global next_id
    if priority not in ["low", "medium", "high"]:
        raise ValueError("Invalid priority")
    task = {
        "id": next_id,
        "title": title,
        "priority": priority,
        "completed": False
    }
    tasks.append(task)
    next_id += 1
    return task


@mcp.tool()
def list_tasks() -> list:
    """Return all tasks."""
    return tasks


@mcp.tool()
def complete_task(task_id: int) -> dict:
    """Mark a task as completed."""
    for task in tasks:
        if task["id"] == task_id:
            task["completed"] = True
            return task
    raise ValueError("Task not found")


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Step 3: Understand the code

Code	Meaning
`FastMCP("Task Server")`	Creates the MCP server
`@mcp.tool()`	Registers a Python function as an MCP tool
`create_task()`	Creates a task
`list_tasks()`	Returns the available tasks
`complete_task()`	Updates a task
`mcp.run(transport="stdio")`	Runs the server using standard input/output transport

The SDK uses the Python function's name, type hints, and docstring to help define the tool and its schema. You do not need to manually implement `tools/list` and `tools/call`; the SDK handles those protocol operations.

Step 4: What tools does the server expose?

When a compatible client connects and discovers the tools, it can find:

```
create_task(title, priority)
list_tasks()
complete_task(task_id)
```

For example, the client can call:

```
create_task(
    title="Prepare for FDE interview",
    priority="high"
)
```

A successful result will resemble:

```
{
  "id": 1,
  "title": "Prepare for FDE interview",
  "priority": "high",
  "completed": false
}
```

This is the basic idea behind turning ordinary Python functions into capabilities an AI application can access.

12. Connecting an AI application to an MCP Server

First, you need an MCP client. The following example demonstrates connecting to the local server, discovering tools, and invoking a tool. Create `task_client.py`:

```
import asyncio
import sys
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


async def main():
    server = StdioServerParameters(
        command=sys.executable,
        args=["task_server.py"]
    )

    async with stdio_client(server) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            # Discover tools
            response = await session.list_tools()
            for tool in response.tools:
                print(tool.name, tool.description)

            # Call a tool
            result = await session.call_tool(
                "create_task",
                arguments={
                    "title": "Prepare for FDE interview",
                    "priority": "high"
                }
            )
            print(result)


asyncio.run(main())
```

Run the client from the same directory:

```
python task_client.py
```

The client starts the server as a subprocess, initializes an MCP session, lists the available tools, and calls `create_task`.

This is a complete MCP client/server demonstration, but it does not yet contain an LLM. To add an LLM, integrate the MCP-discovered tool definitions with an LLM provider's tool-calling interface and implement a dispatcher that forwards the model's chosen tool and arguments to `session.call_tool()`. Exact integration code depends on the model provider and SDK.

13. How the LLM discovers and invokes MCP tools

This is the full end-to-end workflow that an FDE should understand.

1	**User asks a question** "Create a high-priority task to prepare for my interview."
2	**MCP client discovers tools** Calls `tools/list` and obtains the available names, descriptions, and schemas.
3	**Host provides tool definitions to the LLM** The application translates discovered MCP tools into the model provider's tool-calling format.

4	**LLM selects a tool** It requests `create_task` with a title and priority. The LLM is proposing the action, not executing the server code.
5	**Host invokes the MCP tool** The client sends `tools/call` with the selected tool name and arguments.
6	**Server executes the action** It validates the input, creates the task, and returns the result.
7	**LLM produces the final answer** "Done! I created your high-priority interview preparation task."

Important interview clarification

The LLM does not inherently need to speak MCP itself. In a common architecture, the host application uses an MCP client to discover and execute tools, while the LLM uses its own supported tool-calling interface.

> The model's choice is not authorization. Before executing a sensitive operation, the application should check permissions and, where appropriate, request user confirmation.

14. FDE interview revision

These are concise answers to remember.

10 quick questions

1. What is MCP?

Model Context Protocol: an open protocol that standardizes how AI applications connect to external tools, data, and prompts.

2. Why was MCP needed?

Without it, every AI app needed custom integration code for every system. MCP standardizes discovery, inputs, and results so integrations can be reused.

3. What is the difference between an MCP client and server?

The client lives inside the host and maintains a connection to one server. The server exposes tools, resources, and prompts and executes requests against the underlying system.

4. What are tools/list and tools/call?

`tools/list` discovers the tools a server offers (names, descriptions, input schemas). `tools/call` requests execution of a discovered tool.

5. Does MCP replace REST APIs?

No. An MCP tool can wrap an existing REST endpoint, making it easier for AI applications to discover and use.

6. How does an LLM use MCP tools?

The host discovers tools through the MCP client, gives them to the LLM in its tool-calling format, the LLM proposes a call, and the host executes it via `tools/call`.

7. How does MCP help RAG?

A server can expose a tool such as `search_knowledge_base` that queries the vector DB or RAG pipeline and returns relevant passages to the AI.

8. What is the difference between a tool and a resource?

A tool performs an action (do something); a resource provides data or context to read (read something).

9. What is the role of a schema?

It describes the expected structure and types of a tool's inputs, so clients and LLMs know which arguments to provide. It does not guarantee valid model output, so still validate.

10. Is an MCP tool call always safe to execute?

No. The model's choice is not authorization. Check permissions, validate inputs, and request user confirmation for sensitive operations.

What to prioritize for an FDE interview: understand the Host-Client-Server architecture, `tools/list` versus `tools/call`, how the LLM selects tools, how MCP wraps existing APIs and RAG pipelines, and how to build and test a simple server in Python. These topics connect the protocol directly to practical enterprise AI development.